



# 舌尖BOSS — 全国美食探索图鉴 · 技术文档

Author: 卢酉杰 | 软件2403 | 3243012342

Date: 2026-06-14

Description: 本文档详细梳理“舌尖BOSS”项目的完整技术实现，涵盖每个页面的功能、按钮交互、数据结构、算法逻辑及所用到的 HTML / CSS / JavaScript 知识点。

---

## 目录

- [1. 项目整体架构](#)
  - [2. 技术栈全景图](#)
  - [3. 页面逐功能详解](#)
    - [3.1 首页 \(index.html\)](#)
    - [3.2 美食地图页 \(map.html\)](#) ★ 重点
    - [3.3 美食图鉴页 \(list.html\)](#)
    - [3.4 美食详情页 \(detail.html\)](#)
    - [3.5 挑战BOSS页 \(challenge.html\)](#) ★ 重点
  - [4. 核心系统深度剖析](#)
    - [4.1 SVG 地图渲染系统](#)
    - [4.2 智能推荐匹配算法](#)
    - [4.3 LocalStorage 持久化与成就系统](#)
    - [4.4 今日BOSS确定性随机算法](#)
  - [5. 样式系统与设计语言](#)
  - [附录A: 知识要点索引](#)
-

# 1. 项目整体架构

|    |                      |                             |
|----|----------------------|-----------------------------|
| 1  | ms-main/             |                             |
| 2  | ├─ index.html        | # 首页 - 项目入口                 |
| 3  | ├─ map.html          | # 美食地图页 - SVG 交互地图          |
| 4  | ├─ list.html         | # 美食图鉴页 - 多维度筛选列表           |
| 5  | ├─ detail.html       | # 美食详情页 - 单个美食详情            |
| 6  | ├─ challenge.html    | # 挑战BOSS页 - 智能推荐与成就         |
| 7  | ├─ common.css        | # 全局通用样式系统                  |
| 8  | ├─ data.js           | # 美食数据（15道美食） + 辅助映射表       |
| 9  | ├─ china_geo_data.js | # 中国地图 GeoJSON 地理数据（~570KB） |
| 10 | ├─ challenge-core.js | # 挑战系统核心算法（纯逻辑，无DOM操作）      |
| 11 | ├─ challenge.js      | # 挑战页面交互脚本（DOM绑定+事件处理）      |
| 12 | └─ images/logo.png   | # 网站Logo图片                  |

模块职责划分：

| 文件                | 职责                              | 依赖                                         |
|-------------------|---------------------------------|--------------------------------------------|
|                   | 全局样式变量、通用组件<br>（按钮/卡片/标签/导航/页脚） | 无                                          |
| common.css        |                                 |                                            |
|                   | 15道美食数据 + 省份→区域映射 + 省份配色表       | 无                                          |
| data.js           |                                 |                                            |
|                   | 34个省级行政区<br>GeoJSON 边界数据        | 无                                          |
| china_geo_data.js |                                 |                                            |
|                   | 今日BOSS计算、匹配算法、记录管理、成就逻辑         | data.js                                    |
| challenge-core.js |                                 |                                            |
|                   | 挑战页面 DOM 渲染与用户交互                | data.js + challenge-core.js                |
| challenge.js      |                                 |                                            |
|                   | 首页结构 + 内联样式 + 内联脚本              | data.js + challenge-core.js                |
| index.html        |                                 |                                            |
|                   | 地图页结构 + 内联样式 + 内联脚本             | data.js + china_geo_data.js                |
| map.html          |                                 |                                            |
|                   | 图鉴页结构 + 内联样式 + 内联脚本             | data.js                                    |
| list.html         |                                 |                                            |
|                   | 详情页结构 + 内联样式 + 内联脚本             | data.js                                    |
| detail.html       |                                 |                                            |
|                   | 挑战页结构 + 内联样式                    | data.js + challenge-core.js + challenge.js |
| challenge.html    |                                 |                                            |

## 2. 技术栈全景图

### 2.1 HTML5

| 知识点                                                                | 应用位置                              | 功能说明                                                                                                                       |
|--------------------------------------------------------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| 语义化标签                                                              | 所有页面                              | <code>&lt;header&gt;</code> , <code>&lt;nav&gt;</code> , <code>&lt;section&gt;</code> , <code>&lt;footer&gt;</code> 构建页面骨架 |
| <code>&lt;a&gt;</code> 锚点导航                                        | 所有页面导航栏                           | <code>href</code> 实现4个页面间的路由跳转                                                                                             |
| <code>&lt;img&gt;</code> 图片                                        | 所有页面                              | 美食图片展示, <code>onerror</code> 属性和 <code>loading</code> 属性                                                                   |
| <code>&lt;input&gt;</code> 表单                                      | 所有页面搜索框                           | 文本输入, <code>placeholder</code> 占位提示                                                                                        |
| <code>&lt;button&gt;</code> 按钮                                     | 所有页面                              | 搜索、筛选、挑战等交互按钮                                                                                                              |
| <code>&lt;link&gt;</code> 外链资源                                     | 所有页面<br><code>&lt;head&gt;</code> | <code>rel="stylesheet"</code> 引入 CSS, <code>rel="icon"</code> 引入网站图标                                                       |
| <code>&lt;script&gt;</code> 脚本加载                                   | 所有页面                              | <code>src</code> 属性加载外部 JS 文件, 执行顺序由引用顺序决定                                                                                 |
| <code>&lt;meta&gt;</code> 视口                                       | 所有页面<br><code>&lt;head&gt;</code> | <code>viewport</code> 设置响应式视口                                                                                              |
| <code>&lt;style&gt;</code> 内联样式                                    | 每个页面                              | 页面特定样式, 与 <code>common.css</code> 配合                                                                                       |
| URL参数传递                                                            | <code>detail.html</code>          | <code>?id=xxx</code> 查询字符串传递美食ID                                                                                           |
| <code>&lt;svg&gt;</code> 矢量图形                                      | <code>map.html</code>             | 动态生成 SVG 标签绘制中国地图                                                                                                          |
| <code>&lt;defs&gt;</code> /<br><code>&lt;linearGradient&gt;</code> | <code>map.html</code>             | SVG 渐变定义, 用于安徽地图的红色渐变                                                                                                      |

## 2.2 CSS3

| 知识点                                      | 应用位置                    | 功能说明                                                                                                |
|------------------------------------------|-------------------------|-----------------------------------------------------------------------------------------------------|
| CSS 自定义属性 (变量)                           | <code>common.css</code> | <code>:root</code> 定义颜色、圆角、阴影、过渡等全局变量                                                               |
| <code>box-sizing: border-box</code>      | <code>common.css</code> | 全局重置盒模型                                                                                             |
| Flexbox 弹性布局                             | 导航栏、按钮组、标签行、统计卡片        | <code>display: flex</code> 实现一维布局与对齐                                                                |
| Grid 网格布局                                | 美食卡片、统计区、挑战卡片、省份网格      | <code>display: grid</code> 实现二维等分布局                                                                 |
| <code>position: sticky</code>            | 导航栏、图鉴控制面板              | 吸顶效果，滚动时固定在顶部                                                                                       |
| <code>position: absolute/relative</code> | 搜索按钮、Tooltip、装饰元素       | 定位控制                                                                                                |
| <code>linear-gradient()</code> 渐变        | Hero区域、卡片头部、按钮、挑战卡片背景   | 背景渐变效果                                                                                              |
| <code>box-shadow</code> 阴影               | 卡片、导航栏、地图容器             | 三个层级阴影营造层次感                                                                                         |
| <code>transition</code> 过渡动画             | 所有可交互元素                 | 悬浮、点击的平滑过渡                                                                                          |
| <code>@keyframes</code> 关键帧动画            | 搜索框滑入、弹跳、脉冲             | <code>anhuiPulse</code> , <code>bounce</code> , <code>searchSlideIn</code> , <code>slideDown</code> |
| <code>backdrop-filter: blur()</code>     | 粘性控制面板、Tooltip          | 毛玻璃背景模糊效果                                                                                           |
| <code>filter</code> 滤镜                   | SVG地图、成就卡片              | <code>drop-shadow</code> , <code>grayscale</code> , <code>brightness</code>                         |
| <code>@media</code> 媒体查询                 | 所有页面                    | 响应式断点: 1024px / 768px / 640px                                                                       |
| <code>::before / ::after</code> 伪元素      | Tooltip三角箭头、页面装饰        | 纯CSS绘制三角和装饰元素                                                                                       |
| <code>scroll-behavior: smooth</code>     | <code>common.css</code> | 页面平滑滚动                                                                                              |
| <code>:hover / :focus</code> 伪类          | 按钮、输入框、卡片               | 交互状态样式                                                                                              |
| <code>display: none/block/flex</code>    | 搜索框切换、无结果提示、省份二级菜单      | 元素显隐控制                                                                                              |

## 2.3 JavaScript (ES6+)

| 知识点                                          | 应用位置                      | 功能说明                                                |
|----------------------------------------------|---------------------------|-----------------------------------------------------|
| 变量声明 <b>var/let/const</b>                    | 所有JS文件                    | 数据用 <b>var</b> 挂全局，状态用 <b>let</b> ，常量用 <b>const</b> |
| 箭头函数 <b>() =&gt; {}</b>                      | 所有页面                      | 事件回调、数组方法参数                                         |
| 模板字符串 <b>`...\${}...`</b>                    | 所有页面                      | 动态生成 HTML                                           |
| 数组方法 <b>filter/map/find/sort/forEach</b>     | 所有页面                      | 数据筛选、渲染、排序                                          |
| <b>Array.from() / Array.isArray()</b>        | challenge-core.js         | 数组创建与类型判断                                           |
| <b>Set</b> 数据结构                              | challenge-core.js         | 去重统计（已挑战美食ID、挑战日期）                                  |
| <b>... 扩展运算符</b>                             | challenge-core.js         | Set 展开为数组                                           |
| <b>JSON.parse/stringify</b>                  | challenge-core.js         | LocalStorage 数据序列化/反序列化                             |
| <b>localStorage API</b>                      | challenge-core.js         | 挑战记录持久化存储                                           |
| <b>Date</b> 对象                               | challenge-core.js         | 日期计算（今日BOSS、连续天数、记录时间戳）                             |
| <b>Math.abs/round/min</b>                    | challenge-core.js         | 数学运算                                                |
| <b>String.prototype.includes/toLowerCase</b> | 所有页面搜索                    | 字符串模糊匹配                                             |
| <b>DOM API</b>                               | 所有页面                      |                                                     |
| └ document.querySelector/querySelectorAll    | 所有页面                      | 选择器查找DOM元素                                          |
| └ document.getElementById                    | 所有页面                      | ID查找DOM元素                                           |
| └ document.createElement                     | map.html                  | 动态创建DOM（未直接使用，通过innerHTML）                          |
| └ element.innerHTML                          | 所有页面                      | 动态渲染内容                                              |
| └ element.classList.add/remove/toggle        | 所有页面                      | CSS类切换（active/show/mobile-open）                     |
| └ element.setAttribute/getAttribute          | list.html, challenge.html | data属性读写                                            |
| └ element.addEventListener                   | 所有页面                      | 事件监听绑定                                              |
| └ element.scrollToView                       | challenge.js              | 平滑滚动到结果区域                                           |
| └ element.contains                           | 所有页面                      | 判断元素包含关系（点击外部关闭）                                    |
| └ element.getBoundingClientRect              | map.html                  | 获取地图容器位置（Tooltip定位）                                 |
| <b>window.location</b>                       | 所有页面                      | 页面跳转与URL操作                                          |
| └ window.location.href (读/写)                 | 所有页面                      | 页面跳转                                                |
| └ window.location.search                     | detail.html               | 获取URL查询参数                                           |
| <b>URLSearchParams</b>                       | detail.html               | 解析URL查询参数                                           |
| <b>window.open</b>                           | 所有页面                      | 新标签页打开百度搜索                                          |
| <b>history.back()</b>                        | detail.html               | 返回上一页                                               |
| <b>encodeURIComponent</b>                    | 所有页面                      | URL参数编码                                             |
| 事件类型                                         |                           |                                                     |
| └ DOMContentLoaded                           | 所有页面                      | 页面加载完成后的初始化入口                                       |
| └ click                                      | 所有页面                      | 按钮点击、卡片点击、省份点击                                      |
| └ input                                      | index.html, list.html     | 搜索框实时输入监听                                           |
| └ keypress                                   | 所有页面                      | 搜索框回车键监听                                            |
| └ mouseenter/mousemove/mouseleave            | map.html                  | 地图省份悬浮交互                                            |
| └ onerror（内联）                                | 所有页面                      | 图片加载失败时的降级处理                                        |
| <b>event.stopPropagation()</b>               | index.html                | 阻止事件冒泡（卡片内按钮点击）                                     |

### 3. 页面逐功能详解

#### 3.1 首页 (index.html)

首页是项目的门户入口，集成了搜索、数据统计、今日BOSS预览和快速导航。

##### 3.1.1 页面组成结构

|    |                      |                     |
|----|----------------------|---------------------|
| 1  |                      |                     |
| 2  | 导航栏 (Navbar)         | ← 吸顶定位，含导航链接 + 搜索   |
| 3  |                      |                     |
| 4  | Hero 区域 (hero)       | ← 渐变背景 + 标题 + 两个按钮  |
| 5  |                      |                     |
| 6  | 搜索模块 (search)        | ← 输入框 + 搜索按钮 + 动态结果 |
| 7  |                      |                     |
| 8  | 统计卡片 (stats-grid)    | ← 4列网格：大区/美食/口味/城市  |
| 9  |                      |                     |
| 10 | 挑战BOSS区域 (challenge) | ← 3列卡片：今日BOSS/进度/推荐 |
| 11 |                      |                     |
| 12 | 快速导航 (quick-links)   | ← 3列卡片：地图/图鉴/挑战     |
| 13 |                      |                     |
| 14 | 页脚 (footer)          |                     |
| 15 |                      |                     |

##### 3.1.2 逐功能详述

###### 🔴 功能 1：导航栏 (<header class="navbar">)

知识点： CSS `position: sticky` + Flexbox + CSS过渡动画 + JS事件委托 + 事件冒泡

| 元素                             | 功能             | 实现逻辑                                                                                            | 用到知识点                                                            |
|--------------------------------|----------------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Logo区域 .nav-logo               | 点击返回首页         | <code>&lt;a href="index.html"&gt;</code> 超链接                                                    | HTML <code>&lt;a&gt;</code> 标签                                   |
| 导航链接 ×4 .nav-link              | 首页/地图/图鉴/挑战 跳转 | <code>&lt;a href="..."&gt;</code> 超链接；当前页 <code>active</code> 样式加红色                             | CSS <code>:hover</code> 伪类 + 下划线动画                               |
| 搜索图标按钮<br>#navSearchToggle (🔍) | 点击展开/收起搜索框     | <code>classList.toggle('active')</code> → 触发 <code>searchSlideIn</code> CSS动画                   | JS <code>classList.toggle</code> + CSS <code>@keyframes</code>   |
| 搜索输入框<br>#navSearchInput       | 输入关键词          | 支持回车键触搜，聚焦时宽度从240px→280px                                                                       | CSS <code>:focus</code> 伪类 + JS <code>keypress</code> 事件         |
| 搜索提交按钮<br>#navSearchSubmit (→) | 跳转百度搜索         | <code>window.open('https://www.baidu.com/s?wd=' + encodeURIComponent(keyword + ' 美食'))</code>   | <code>window.open</code> + <code>encodeURIComponent</code> ，外部搜索 |
| 手机菜单按钮<br>#mobileMenuBtn (☰)   | 移动端展开导航        | <code>classList.toggle('mobile-open')</code> → 导航变为纵向列表                                         | CSS <code>@media</code> 响应式 + 绝对定位下拉菜单                           |
| 点击外部关闭                         | 点击搜索框外 → 收起    | <code>document.addEventListener('click', ...)</code> 判断 <code>element.contains(e.target)</code> | 事件委托 + 事件冒泡                                                      |

## 📌 功能 2: Hero 区域

知识点: CSS 渐变背景 + Flexbox居中 + `<a>` 按钮

| 元素                              | 功能                        | 实现逻辑                                                            | 用到知识点                                                  |
|---------------------------------|---------------------------|-----------------------------------------------------------------|--------------------------------------------------------|
| 徽章 <code>.hero-badge</code>     | 显示 "🏆 全国美食挑战图鉴"           | 内联块元素 + 圆角背景                                                    | CSS <code>display: inline-block + border-radius</code> |
| 大标题 <code>.hero-title</code>    | "探索中国美食文化"                | 3.5rem 大字号 800字重                                                | CSS <code>font-weight: 800 + line-height: 1.2</code>   |
| 副标题 <code>.hero-subtitle</code> | 两行宣传语                     | <code>&lt;br&gt;</code> 换行                                      | HTML <code>&lt;br&gt;</code> 标签                        |
| "查看地图"按钮                        | 跳转 <code>map.html</code>  | <code>&lt;a href="map.html" class="btn btn-primary"&gt;</code>  | CSS按钮系统: <code>btn-primary</code> = 红色填充 + 悬浮上移 + 阴影   |
| "浏览图鉴"按钮                        | 跳转 <code>list.html</code> | <code>&lt;a href="list.html" class="btn btn-outline"&gt;</code> | CSS按钮系统: <code>btn-outline</code> = 透明+红色边框 → 悬浮填充红色   |

按钮系统知识点总结:

- `btn-primary`: `background: var(--accent-primary)` + `:hover` 加深色+上移 2px+阴影
- `btn-outline`: `background: transparent` + `border: 2px solid var(--accent-primary)` + `:hover` 填充背景
- 使用 CSS 变量 `var(--accent-primary)` 统一品牌色

## 📌 功能 3: 本地搜索模块

知识点: JavaScript `Array.filter()` + `Event: input` (实时搜索) + `Event: keypress` (回车搜索) + 模板字符串 + DOM动态渲染

数据流:

```
1 用户输入 → input/keypress/click事件 → performSearch()
2  → foodBosses.filter(匹配算法) → 动态生成HTML → innerHTML渲染
```

| 按钮/元素                 | 功能          | 背后实现                                                                      |
|-----------------------|-------------|---------------------------------------------------------------------------|
| 搜索输入框<br>#searchInput | 实时搜索        | 监听 input 事件：长度≥1 → 自动执行搜索；长度=0 → 隐藏结果                                     |
| 搜索按钮<br>#searchBtn    | 手动触发<br>搜索  | 监听 click 事件 → performSearch()                                             |
| 回车键                   | 键盘触发<br>搜索  | 监听 keypress 事件 → e.key === 'Enter'                                        |
| 搜索结果项                 | 点击跳转<br>详情页 | onclick="window.location.href='detail.html?id=\${food.id}'" — 通过URL传递ID参数 |

匹配逻辑 (`performSearch` 函数):

```
1  const results = foodBosses.filter(food => {
2      return food.name.toLowerCase().includes(keyword) || // 美食名
3             food.city.toLowerCase().includes(keyword) || // 城市
4             food.region.toLowerCase().includes(keyword) || // 区域
5             food.type.toLowerCase().includes(keyword) || // 口味类型
6             food.taste.some(t => t.toLowerCase().includes(keyword)) ||
7             // 口味标签 (数组)
8             food.desc.toLowerCase().includes(keyword); // 简介
9  });
```

涉及知识点:

- `Array.filter()` — 高阶函数过滤
- `String.toLowerCase()` — 大小写不敏感匹配
- `String.includes()` — ES6字符串包含判断
- `Array.some()` — 数组中任一元素满足条件
- 模板字符串 ``...${}...`` — 动态HTML生成

## 🔴 功能 4: 数据统计卡片

知识点: CSS Grid 网格布局 + JS动态DOM更新



| 统计项           | 值来源                            | JS实现                                                                              |
|---------------|--------------------------------|-----------------------------------------------------------------------------------|
| 地理大区 硬编码 "7"  |                                | 静态HTML                                                                            |
| 特色美食          | <code>foodBosses.length</code> | <code>document.getElementById('foodCount').textContent = foodBosses.length</code> |
| 口味类型 硬编码 "8"  |                                | 静态HTML                                                                            |
| 代表城市 硬编码 "12" |                                | 静态HTML                                                                            |

布局特点: `grid-template-columns: repeat(4, 1fr)` — 4列等宽, 移动端 `@media (max-width: 768px)` 变为2列。

## 📌 功能 5: 今日BOSS预览卡片区域 ★

知识点: 函数调用 + 动态DOM生成 + CSS Grid 2:1:1布局 + 事件冒泡控制

三列布局:

|   |              |                                                                                    |                                                                                    |
|---|--------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| 1 |              |                                                                                    |                                                                                    |
| 2 | 今日BOSS (2fr) | 进度 (1fr)                                                                           | 推荐 (1fr)                                                                           |
| 3 | [图片]         |  |  |
| 4 | 美食名称         | 已挑战 x/15                                                                           | 智能推荐                                                                               |
| 5 | 描述           | 进度条                                                                                | 描述文字                                                                               |
| 6 | [接受挑战] 按钮    | 查看详情 →                                                                             | [开始挑战]                                                                             |
| 7 |              |                                                                                    |                                                                                    |

| 卡片/按钮                                         | 功能       | 实现细节                                                                                               |
|-----------------------------------------------|----------|----------------------------------------------------------------------------------------------------|
| 主卡片 (今日BOSS)                                  | 展示当日BOSS | <code>initTodayBossPreview()</code> 调用 <code>getTodayBoss()</code> , 动态渲染图片+名称+描述+按钮               |
| 主卡片点击                                         | 跳转挑战页    | <code>onclick="window.location.href='challenge.html'"</code>                                       |
| "接受挑战"按钮                                      | 跳转挑战页    | 主卡片的子按钮, 同样跳转                                                                                      |
| 进度卡片                                          | 显示挑战进度   | <code>initChallengeProgress()</code> 调用 <code>getChallengeStats()</code> , 显示 x/15 + 进度条百分比        |
| 进度条 <code>.challenge-card-progress-bar</code> | 可视化进度    | <code>style.width = \${stats.percentage}%,</code> CSS过渡动画 <code>transition: width 0.3s ease</code> |
| "查看详情 →"链接                                    | 跳转挑战页    | <code>&lt;a href="challenge.html"&gt;</code>                                                       |
| 推荐卡片                                          | 智能推荐入口   | 静态文字描述                                                                                             |
| "开始挑战"按钮                                      | 跳转挑战页    | <code>event.stopPropagation()</code> 阻止冒泡后跳转                                                       |

关键知识点: `event.stopPropagation()` — 推荐卡片整体可点击跳转, 但内部的"开始挑战"按钮需要先 `stopPropagation` 阻止事件冒泡到父级卡片, 避免触发两次跳转。

## 📌 功能 6: 快速导航区

知识点: CSS Grid 3列 + 悬浮动画 + `<a>` 整卡可点

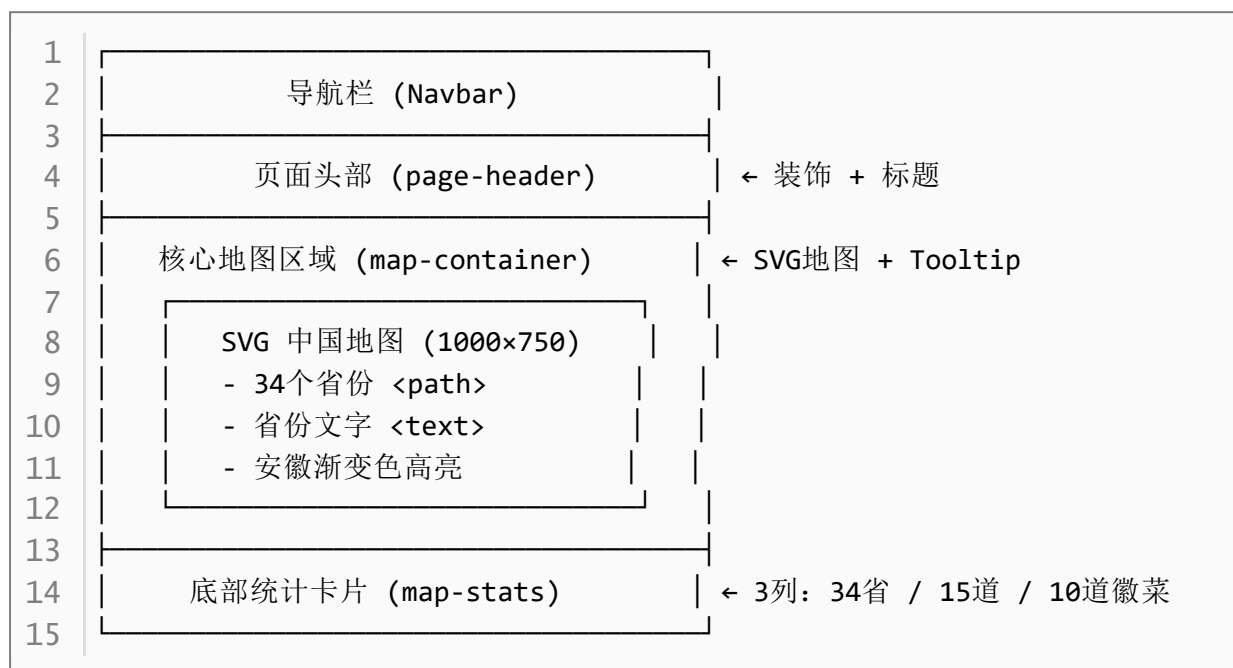
| 卡片                                                                                          | 点击跳转                        | 悬浮效果                                                         |
|---------------------------------------------------------------------------------------------|-----------------------------|--------------------------------------------------------------|
| 美食地图<br>   | <code>map.html</code>       | <code>translateY(-6px) + box-shadow: var(--shadow-lg)</code> |
| 美食图鉴<br>   | <code>list.html</code>      | 同上                                                           |
| 挑战BOSS<br> | <code>challenge.html</code> | 同上                                                           |

整张卡片都是 `<a>` 标签包裹, 实现整卡可点的效果。

## 3.2 美食地图页 (`map.html`) ★ 重点

地图页是项目技术含金量最高的页面, 使用 SVG 自绘中国地图, 实现省份交互。

### 3.2.1 页面架构



### 3.2.2 SVG地图渲染核心流程 ★

核心知识点： GeoJSON → 坐标投影 → SVG Path → DOM动态注入

完整渲染管线：

```
1 china_geo_data.js (GeoJSON) → initMap() → buildMap()
2   └─ 1. 定义投影参数 (地图边界 + SVG画布大小)
3   └─ 2. project(lng, lat) 经纬度 → SVG坐标 (线性映射)
4   └─ 3. geoToPath(geometry) 坐标点 → SVG d属性 (M/L/Z命令)
5   └─ 4. 为每个省份生成：
6       └─ <path> (填充区域)
7       └─ <text> (省份名称标签)
8   └─ 5. innerHTML 注入容器
9   └─ 6. bindEvents() 绑定交互事件
```

#### ① 数据源 (`china_geo_data.js`):

- 格式：`var CHINA_GEO = { "type": "FeatureCollection", "features": [...] }`
- 每个 feature 包含：
  - `properties.name`: 省份名称
  - `properties.centroid`: 视觉中心点坐标 `[lng, lat]`
  - `geometry`: 边界坐标 (类型为 `Polygon` 或 `MultiPolygon`)

#### ② 坐标投影算法 (`project` 函数):

```
1 function project(lng, lat) {
2   const x = ((lng - MAP_LNG_MIN) / (MAP_LNG_MAX - MAP_LNG_MIN)) *
MAP_W;
3   const y = ((MAP_LAT_MAX - lat) / (MAP_LAT_MAX - MAP_LAT_MIN)) *
MAP_H;
4   return [x.toFixed(2), y.toFixed(2)];
5 }
```

知识点：线性映射 (Linear Mapping)

- 中国经度范围：73° E ~ 136° E → 映射到 0 ~ 1000 (SVG宽度)
- 中国纬度范围：17° N ~ 54° N → 映射到 750 ~ 0 (SVG高度, Y轴反转)

- 公式：
$$\text{output} = (\text{input} - \text{input\_min}) / (\text{input\_max} - \text{input\_min}) * \text{output\_range}$$
- 纬度需要反转（中国最北54° 在SVG Y轴最小处）

### ③ SVG 路径生成 (`geoToPath` 函数):

```
1 function geoToPath(geometry) {
2   // 处理 Polygon 和 MultiPolygon 两种类型
3   // 使用 SVG 路径命令:
4   //   M x,y = MoveTo (移动到起点)
5   //   L x,y = LineTo (直线到点)
6   //   Z      = ClosePath (闭合路径)
7   // 生成格式: "M x1,y1 L x2,y2 L x3,y3 ... Z"
8 }
```

知识点: SVG Path 微语言

- **M** (MoveTo) — 不画线, 只移动笔触
- **L** (LineTo) — 从当前点画直线到目标点
- **Z** (ClosePath) — 闭合当前路径
- 省份形状的复杂度决定了路径中坐标点数量 (约数百到数千个点)

### ④ SVG 渐变定义 (安徽省高亮):

```
1 <defs>
2   <linearGradient id="anhuiGradient" x1="0%" y1="0%" x2="100%"
3   y2="100%">
4     <stop offset="0%" style="stop-color:#E63946;stop-opacity:1" />
5     <stop offset="100%" style="stop-color:#9B2226;stop-opacity:1"
6   />
7   </linearGradient>
8 </defs>
```

知识点: SVG `<linearGradient>` — 定义线性渐变, 通过 `url(#anhuiGradient)` 引用

### ⑤ 省份标签文字描边:

```

1  .map-label {
2      paint-order: stroke fill;          /* 先描边后填充 */
3      stroke: rgba(255, 255, 255, 0.85); /* 白色描边 */
4      stroke-width: 2.5px;               /* 描边宽度 */
5      stroke-linejoin: round;            /* 描边圆角连接 */
6  }

```

知识点: `paint-order` — 控制SVG元素绘制顺序, `stroke fill` 表示先画描边再画填充, 实现文字描边效果 (文字在任意颜色背景上都清晰可读)

### 3.2.3 地图交互系统 ★

核心知识点: DOM事件 (`mouseenter/mousemove/mouseleave/click`)  
+ `getBoundingClientRect` + CSS `position: absolute`

| 交互                  | 触发事件                    | 实现逻辑                                                                                                 |
|---------------------|-------------------------|------------------------------------------------------------------------------------------------------|
| 悬浮省份 → 高亮           | <code>mouseenter</code> | 省份 <code>&lt;path&gt;</code> 的CSS <code>:hover</code> 自动触发 ( <code>fill-opacity: 0.85</code> + 金色描边) |
| 悬浮省份 →<br>Tooltip显示 | <code>mouseenter</code> | <code>tooltip.classList.add('show')</code> , 更新省份名称和美食数量                                             |
| 鼠标移动 →<br>Tooltip跟随 | <code>mousemove</code>  | <code>e.clientX - mapContainer.getBoundingClientRect().left</code><br>计算相对位置                         |
| 移出省份 →<br>Tooltip消失 | <code>mouseleave</code> | <code>tooltip.classList.remove('show')</code>                                                        |
| 悬浮省份 → 标签<br>放大     | <code>mouseenter</code> | <code>labelElement.classList.add('hovered')</code> → 字号14px、红色                                       |
| 点击省份                | <code>click</code>      | <code>window.open('https://www.baidu.com/s?wd=省份+特色美食')</code><br>外部搜索                               |

Tooltip 定位原理:

```

1  path.addEventListener('mousemove', function(e) {
2      const rect = mapContainer.getBoundingClientRect(); // 获取地图容器
      相对视口位置
3      const x = e.clientX - rect.left;                  // 鼠标相对容
      器X坐标
4      const y = e.clientY - rect.top;                  // 鼠标相对容
      器Y坐标
5      tooltip.style.left = x + 'px';
6      tooltip.style.top = y + 'px';
7  });

```

知识点: `element.getBoundingClientRect()` — 返回元素相对于视口的位置和尺寸, `e.clientX/clientY` 是鼠标相对于视口的坐标, 两者相减得到相对于容器的坐标。

### Tooltip CSS 三角箭头 (纯CSS):

```
1 .map-tooltip::after {  
2   content: "";  
3   border-width: 8px 8px 0;  
4   border-style: solid;  
5   border-color: rgba(255, 255, 255, 0.95) transparent transparent;  
6 }
```

知识点: CSS 三角形技巧 — 利用边框交汇处的斜角, 设置一边有颜色其余透明, 即形成三角形箭头。

### 3.2.4 每个按钮详述

| 按钮          | 知识点                                                            | 说明                 |
|-------------|----------------------------------------------------------------|--------------------|
| Logo        | <a> 标签                                                         | 返回首页               |
| 导航链接×4      | <a> 标签 + CSS :hover                                            | 页面跳转 + 悬浮变色        |
| 搜索图标 (🔍)    | JS <code>classList.toggle</code> + CSS <code>@keyframes</code> | 展开导航搜索框 (滑入动画)     |
| 搜索输入框       | JS <code>keypress</code> 事件 / <code>:focus</code> 伪类           | 回车搜索 / 聚焦加宽        |
| 搜索提交 (→)    | <code>window.open</code> + <code>encodeURIComponent</code>     | 百度外部搜索             |
| 手机菜单 (☰)    | JS <code>classList.toggle</code> + CSS <code>@media</code>     | 移动端展开导航            |
| 省份 <path>   | <code>mouseenter/mousemove/mouseleave/click</code>             | 悬浮Tooltip + 点击百度搜索 |
| 省份标签 <text> | 联动 <code>mouseenter/mouseleave</code>                          | 悬浮时放大变红            |

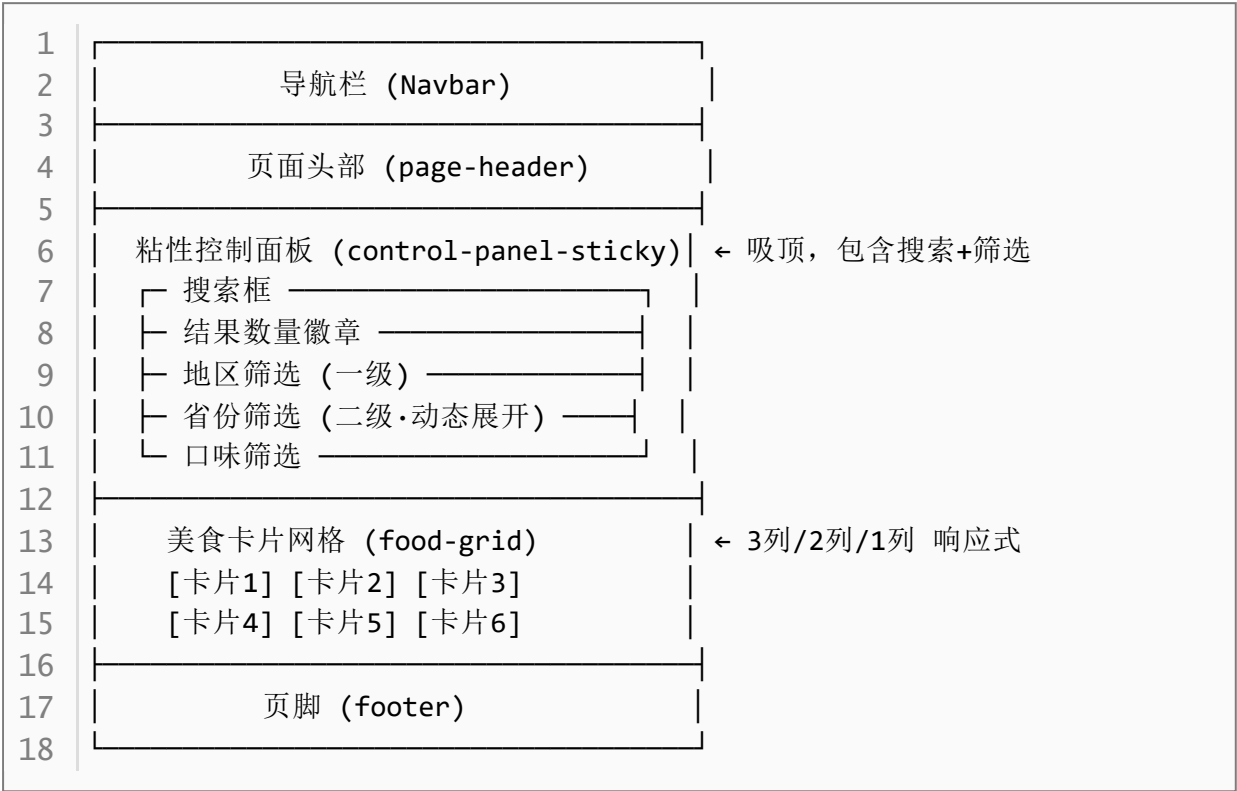
3.2.5 安徽省特殊处理汇总

| 处理项           | 技术手段                                                                 |
|---------------|----------------------------------------------------------------------|
| 渐变色填充         | SVG <linearGradient id="anhuiGradient"> → fill="url(#anhuiGradient)" |
| 脉冲动画          | CSS @keyframes anhuiPulse → drop-shadow 大小交替                         |
| 较粗金色描边        | .anhui-highlight { stroke-width: 2.5; stroke: #FFD166; }             |
| 标签白字深描边       | .anhui-label { fill: #FFFFFF; stroke: #9B2226; font-size: 14px; }    |
| 省份名"安徽省"→"安徽" | shortName = name.replace(/(省 市 ...)/g, '') 正则替换                      |

3.3 美食图鉴页 (list.html)

图鉴页提供多维度筛选和实时搜索的美食列表浏览。

3.3.1 页面架构



### 3.3.2 逐功能详述

---

#### 📌 功能 1：粘性控制面板

知识点： CSS `position: sticky` + `backdrop-filter: blur()`（毛玻璃） + `z-index` 层叠管理

```
1 .control-panel-sticky {
2   position: sticky;
3   top: 57px;           /* 导航栏下方 */
4   z-index: 150;
5   background: rgba(255, 255, 255, 0.85);
6   backdrop-filter: blur(20px);      /* 毛玻璃模糊 */
7   -webkit-backdrop-filter: blur(20px); /* Safari兼容 */
8 }
```

知识点详解：

- `position: sticky` — “粘性定位”，在滚动到阈值前正常流布局，滚动过阈值后变为固定定位
- `top: 57px` — 导航栏高度约为57px，控制面板在导航栏下方吸顶
- `backdrop-filter: blur(20px)` — 对面板后面的内容进行高斯模糊，实现毛玻璃效果

---

#### 📌 功能 2：实时搜索

知识点： `input` 事件 + `Array.filter()` 多字段匹配

与首页搜索相同逻辑，但增加了 `food.province` 字段的匹配。

特殊UI元素： 结果数量徽章 `#resultCount` — 显示当前筛选条件下的美食总数，使用红色胶囊样式。

---

#### 📌 功能 3：地区筛选（一级）



知识点： CSS类切换 + `data-*` 自定义属性 + 事件绑定

| 按钮      | <code>data-region</code> | 点击行为                                                |
|---------|--------------------------|-----------------------------------------------------|
| 全部      | 全部                       | <code>currentRegion = '全部' → 重新渲染 + 隐藏省份二级菜单</code> |
| 华北      | 华北                       | <code>currentRegion = '华北' → 展开省份二级菜单</code>        |
| 东北 ~ 西北 | 对应大区                     | 同上                                                  |

CSS 胶囊按钮激活状态切换：

```
1 document.querySelectorAll('#regionFilters .filter-btn').forEach(btn =>
  {
2   btn.addEventListener('click', function() {
3     // 1. 移除所有按钮的 active 类
4     document.querySelectorAll('#regionFilters .filter-btn')
5       .forEach(b => b.classList.remove('active'));
6     // 2. 为当前按钮添加 active 类（红色渐变背景）
7     this.classList.add('active');
8     // 3. 更新筛选状态
9     currentRegion = this.getAttribute('data-region');
10    // 4. 触发界面更新
11    updateProvinceFilters();
12    renderFoodCards();
13  });
14 });
```

CSS 激活样式：

```
1 .filter-btn.active {
2   background: linear-gradient(135deg, #FF5A5A 0%, #E63946 100%);
3   color: white;
4   box-shadow: 0 3px 10px rgba(230, 57, 70, 0.2);
5 }
```

#### 📌 功能 4：省份筛选（二级联动）

知识点： 条件渲染 + 动态DOM + `data-*` 属性 + 数据驱动

数据结构 — 大区→省份映射：

```
1 const regionProvinces = {  
2   '华北': ['北京市', '天津市', '河北省', '山西省', '内蒙古自治区'],  
3   '华东': ['上海市', '江苏省', '浙江省', '安徽省', '福建省', '江西省',  
4     '山东省', '台湾省'],  
5   // ... 其他大区  
};
```

省份二级菜单展开逻辑：

1. 当 `currentRegion !== '全部'` → 显示省份二级菜单
2. 遍历该大区下所有省份 → 生成胶囊按钮
3. 每个按钮显示省份简称 + 美食数量（如 "安徽 (10)"）
4. 无美食的省份 → `disabled` 类 + `disabled` 属性

展开动画：

```
1 @keyframes slideDown {  
2   from { opacity: 0; transform: translateY(-6px); }  
3   to   { opacity: 1; transform: translateY(0); }  
4 }  
5 .province-filters.show {  
6   display: block;  
7   animation: slideDown 0.25s cubic-bezier(0.16, 1, 0.3, 1) forwards;  
8 }
```

知识点：CSS `cubic-bezier()` 缓动函数 — 自定义动画速率曲线，  
(0.16, 1, 0.3, 1) 实现类似 `easeOut` 的回弹效果。

---

## 🔴 功能 5：口味筛选

知识点： 同类筛选模式（与地区筛选共享同一套逻辑模式）

| 按钮  | 口味类型              | 预置美食数 |
|-----|-------------------|-------|
| 全部  | —                 | 15    |
| 麻辣系 | 重庆火锅              | 1     |
| 精致系 | 广式早茶              | 1     |
| 经典系 | 臭鳊鱼/烧鸡/板鸭/一品锅/烤鸭  | 5     |
| 面食系 | 烧饼/虾子面/格拉条/酥饼/牛肉面 | 5     |
| 硬核系 | 肉夹馍               | 1     |
| 重口系 | 毛豆腐               | 1     |
| 鲜香系 | 牛肉汤               | 1     |

## 📌 功能 6：美食卡片网格

知识点：CSS Grid 响应式 + 图片加载降级 + `onerror` 处理

响应式断点：

```
1 .food-grid { grid-template-columns: repeat(3, 1fr); } /* 桌面：3列 */
2 @media (max-width: 1024px) { grid-template-columns: repeat(2, 1fr); } /* 平板：2列 */
3 @media (max-width: 640px) { grid-template-columns: 1fr; } /* 手机：1列 */
```

图片加载降级处理：

```
1 <img onerror="this.style.display='none';
  this.nextElementSibling.style.display='flex';">
2 <div class="food-card-icon-fallback" style="display:none;">🍜</div>
```

知识点：`onerror` 事件 + `nextElementSibling` — 图片加载失败时隐藏img，显示相邻的Emoji备选图标

| 卡片/按钮 | 功能      | 知识点                                                                              |
|-------|---------|----------------------------------------------------------------------------------|
| 整张卡片  | 点击跳转详情页 | <code>onclick="window.location.href='detail.html?id=\${food.id}'"</code> — URL传参 |
| 悬浮效果  | 上移 + 阴影 | CSS transition + <code>:hover</code>                                             |

## 📌 功能 7：筛选数据流

```
1 搜索输入 → searchKeyword 更新
2 地区点击 → currentRegion 更新 (可能触发省份菜单刷新)
3 省份点击 → currentProvince 更新
4 口味点击 → currentType 更新
5     ↓
6   renderFoodCards() ← 4个条件组合 filter
7     ↓
8   foodBosses.filter(food => { 搜索匹配 && 地区匹配 && 省份匹配 && 口味
   匹配 })
9     ↓
10  innerHTML 重绘卡片 + 更新数量徽章
```

### 3.4 美食详情页 (detail.html)

详情页展示单个美食的完整信息，通过URL参数获取美食ID。

#### 3.4.1 页面结构

```
1  ┌────────────────────────────────────────────────────────────────────────────────┐
2  │                                     导航栏 (Navbar)                             │
3  └────────────────────────────────────────────────────────────────────────────────┘
4  ┌────────────────────────────────────────────────────────────────────────────────┐
5  │                                     ← 返回上一页                               │
6  └────────────────────────────────────────────────────────────────────────────────┘
7  ┌────────────────────────────────────────────────────────────────────────────────┐
8  │  ┌──────────────────────────────────────────────────────────────────────────┐
9  │  │  详情卡片 (detail-card)
10 │  │  ┌──────────────────────────────────────────────────────────────────┐
11 │  │  │  头部 (detail-header)
12 │  │  │  - 高清图片 (800×400)
13 │  │  │  - 美食名称 (2.5rem)
14 │  │  │  - 城市·省份
15 │  │  │  - 口味标签
16 │  │  └──────────────────────────────────────────────────────────────────┐
17 │  │  正文 (detail-body)
18 │  │  - 基本信息 (2×2 Grid)
19 │  │  │  · 所属省份 · 代表城市
20 │  │  │  · 美食等级 · 口味类型
21 │  │  - 美食简介
22 │  │  - 详细介绍 (历史/工艺)
23 │  └──────────────────────────────────────────────────────────────────────────┐
24 │  └──────────────────────────────────────────────────────────────────────────┐
25 │  ┌──────────────────────────────────────────────────────────────────────────┐
26 │  │  [浏览更多美食] [查看美食地图]
27 │  └──────────────────────────────────────────────────────────────────────────┐
28 │  ┌──────────────────────────────────────────────────────────────────────────┐
29 │  │  页脚 (footer)
30 │  └──────────────────────────────────────────────────────────────────────────┐
31 └────────────────────────────────────────────────────────────────────────────────┘
```

### 3.4.2 核心实现

---

#### 📌 功能 1: URL参数解析获取美食ID

知识点: `URLSearchParams` API + `parseInt` + `Array.find()`

```
1 const urlParams = new URLSearchParams(window.location.search); // 解析
  ?id=1
2 const foodId = parseInt(urlParams.get('id')); // 获取
  id参数 → 整数
3 const food = foodBosses.find(f => f.id === foodId); // 查找
  匹配美食
```

知识点详解:

- `window.location.search` — 获取当前URL的查询字符串（如 `?id=3`）
- `URLSearchParams` — ES6 API，解析查询字符串为键值对
- `.get('id')` — 获取指定参数的值（字符串）
- `parseInt()` — 字符串转整数
- `Array.find()` — ES6，返回第一个满足条件的元素

---

#### 📌 功能 2: 404 处理

```
1 if (!food) {
2   document.getElementById('detailCard').innerHTML = `
3     <div class="detail-header">
4       <h1 class="detail-name">未找到该美食</h1>
5       <p class="detail-location">请返回重新选择</p>
6     </div>`;
7 }
```

知识点: 防御性编程 — URL传入不存在的ID时给出友好提示

---

### 📌 功能 3：图片降级处理

```
1 if (food.img) {  
2     imgHtml = `<img ... onerror="this.onerror=null;  
this.src='...placeholder...';">`;   
3 }
```

知识点： **this.onerror=null** — 防止备用图片也加载失败时的无限错误循环

| 按钮       | 功能     | 知识点                                                                                 |
|----------|--------|-------------------------------------------------------------------------------------|
| ← 返回上一頁  | 返回之前一頁 | <code>&lt;a href="javascript:history.back()"&gt;</code> — <b>history.back()</b> API |
| "浏览更多美食" | 跳转图鉴頁  | <code>&lt;a href="list.html" class="btn btn-outline"&gt;</code>                     |
| "查看美食地图" | 跳转地图頁  | <code>&lt;a href="map.html" class="btn btn-primary"&gt;</code>                      |

## 3.5 挑战BOSS页 (challenge.html) ★ 重点

挑战页是项目交互逻辑最复杂的页面，包含今日BOSS、四维智能推荐、挑战记录和成就系统。

### 3.5.1 页面架构





```

1 const moodHtml = MOODS.map(m => `
2   <div class="option-btn" data-type="mood" data-value="${m.id}">
3     <div class="option-emoji">${m.emoji}</div>
4     <div class="option-label">${m.label}</div>
5   </div>
6 `).join('');
7 document.getElementById('moodOptions').innerHTML = moodHtml;

```

选项点击交互（事件委托模式）：

```

1 document.querySelectorAll('.option-btn').forEach(btn => {
2   btn.addEventListener('click', function() {
3     const type = this.getAttribute('data-type'); //
4     'mood'/'weather'/'scene'/'taste'
5     const value = this.getAttribute('data-value'); // 对应的id
6
7     // 同类型其他按钮取消选中
8     document.querySelectorAll(`.option-btn[data-type="${type}"]`)
9       .forEach(b => b.classList.remove('active'));
10
11    // 当前按钮选中
12    this.classList.add('active');
13
14    // 更新状态对象
15    selectedConditions[type] = value;
16  });
17 });

```

知识点：属性选择器 ``[data-type="${type}"]`` — CSS选择器结合模板字符串，精确选中同类按钮



### 3.5.3 每个按钮详述

| 按钮                   | 功能       | 触发函数                                  | 分支逻辑                                                                      |
|----------------------|----------|---------------------------------------|---------------------------------------------------------------------------|
| "✖ 接受今日挑战"           | 接受当日BOSS | <code>acceptTodayBoss()</code>        | 自动匹配打分 → 保存记录 → 弹窗 → 刷新                                                   |
| "今日已挑战 ✓" (disabled) | 提示已完成    | —                                     | 按钮 <code>disabled</code> 属性，样式为灰色                                         |
| 心情/天气/场景/口味选项        | 选择维度     | 内联 <code>click</code> 事件              | <code>classList.add('active')</code> + 更新 <code>selectedConditions</code> |
| "为我推荐"               | 智能推荐     | <code>generateRecommendation()</code> | 检查选择完整性 → 计算匹配 → 渲染结果 → 滚动到结果区                                            |
| "✓ 接受挑战"             | 接受推荐结果   | <code>acceptRecommendation()</code>   | 保存记录 → 弹窗 → 刷新统计 → 隐藏结果 → 重置选择                                            |
| "🔄 换一个"              | 换下一个推荐   | <code>retryRecommendation()</code>    | 取排序后的下一个 → 重新渲染                                                           |

### 3.5.4 今日BOSS接受挑战流程

```
1  用户点击 "✖ 接受今日挑战"
2      ↓
3  acceptTodayBoss() 执行
4      ↓
5  getTodayBoss() → 获取今日BOSS
6  calculateMatch() → 自动匹配打分 (使用默认条件)
7  saveChallengeRecord() → 写入 LocalStorage
8  showSuccessModal() → alert弹窗显示结果
9      ↓
10 刷新三个区域:
11  initTodayBoss() → 按钮变为 "今日已挑战 ✓" (disabled)
12  initStats() → 更新统计数字
13  initAchievements() → 检查/解锁新成就
```

## 4. 核心系统深度剖析

### 4.1 SVG 地图渲染系统

#### 4.1.1 数据流全链路

```
1  【数据层】china_geo_data.js
2  CHINA_GEO = {
3      type: "FeatureCollection",      ← GeoJSON 标准格式
4      features: [
5          {
6              type: "Feature",
```

```

7     properties: {
8         name: "安徽省",           ← 省份名称
9         centroid: [117.17, 31.86], ← 视觉中心经纬度
10        center: [117.17, 31.86]   ← 备用中心点
11    },
12    geometry: {
13        type: "MultiPolygon",      ← 支持多多边形（含岛屿）
14        coordinates: [[[[lng,lat], ...]]] ← 4层嵌套
15    }
16 },
17 ... (共34个省份)
18 ]
19 }

```

【渲染层】map.html 内联脚本

```

22 initMap()
23   └─ 检查 CHINA_GEO 是否存在（容错）
24   └─ buildMap(CHINA_GEO, container)
25       └─ 定义投影参数
26           └─ MAP_W=1000, MAP_H=750      (SVG画布)
27           └─ LNG 73→136, LAT 17→54    (中国范围)
28       └─ project(lng, lat)              (坐标映射)
29           └─ 线性映射公式:
30               x = (lng-73)/(136-73) * 1000
31               y = (54-lat)/(54-17) * 750 ← Y轴反转
32       └─ geoToPath(geometry)            (路径生成)
33           └─ 处理 Polygon + MultiPolygon
34           └─ 输出 SVG d 属性: "M x,y L x,y ... Z"
35       └─ 构建 SVG 字符串
36           └─ <svg viewBox="0 0 1000 750">
37               └─ <defs> (渐变定义) </defs>
38                   └─ <g id="provinces"> (省份path集合) </g>
39                       └─ <g id="labels"> (省份文字集合) </g>
40                           └─ </svg>
41       └─ container.innerHTML = svg + tooltip
42       └─ bindEvents()                (事件绑定)

```

## 4.1.2 关键技术点

① 多层嵌套坐标处理:

```

1  const processRing = ring => {
2      let d = '';

```

```

3     ring.forEach((point, i) => {
4         const [x, y] = project(point[0], point[1]); // [lng, lat] →
[x, y]
5         d += (i === 0 ? 'M' : 'L') + x + ',' + y;    // 首点用M, 其余
用L
6     });
7     return d + 'Z'; // 闭合路径
8 };
9
10 // MultiPolygon: 多个独立多边形 (如浙江含岛屿)
11 // Polygon: 单个多边形 (含洞时外环+内环)
12 if (geometry.type === 'MultiPolygon') {
13     geometry.coordinates.forEach(polygon => {
14         polygon.forEach(ring => parts.push(processRing(ring)));
15     });
16 }

```

## ② 省份视觉中心微调:

某些省份 (如内蒙古、甘肃) 的几何中心不在视觉最佳位置, 使用偏移字典手动校正:

```

1 const offsetMap = {
2   '河北省': [-0.8, -0.6],
3   '内蒙古自治区': [0, -1.8], // 东西跨度大, 需向北偏移
4   '甘肃省': [1.5, -0.8],      // 形状狭长, 需向右上偏移
5   '广东省': [0, 0.4],         // 雷州半岛向南延伸
6 };

```

## ③ 省份简称处理 (正则替换):

```

1 const shortName = name.replace(
2   /(省|市|维吾尔自治区|回族自治州|壮族自治区|自治区|特别行政区)/g, ''
3 );
4 // "安徽省" → "安徽"
5 // "新疆维吾尔自治区" → "新疆"
6 // "香港特别行政区" → "香港"

```

知识点: 正则表达式 `replace()` — 全局替换 `g` 标志, 管道 `|` 或逻辑

## 4.2 智能推荐匹配算法

### 4.2.1 算法总览

核心知识点： 加权打分算法 + 标签匹配 + 多维条件组合

完整评分模型：

```
1 总分 = 基础分(60) + 口味匹配(15) + 场景匹配(10) + 心情匹配(10) + 天气匹配(5) + 徽菜加分(5)
2 上限 = 100分
```

### 4.2.2 分项匹配逻辑

#### ① 基础分 = 60

每道菜至少60分起步，确保都有基本推荐可能。

#### ② 口味匹配（最高+15分）：

```
1  if (conditions.taste === 'any') {
2      score += 5; // "随便" → +5
3  } else if (tasteTags.includes(food.type)) { // 精确匹配
4      score += 15;
5      reasons.push(`口味完美匹配（${food.type}）`);
6  } else if (条件近似匹配) { // 模糊匹配
7      score += 12;
8  }
```

知识点： `Array.includes()` — 判断口味标签数组中是否包含该美食的类型

#### ③ 场景匹配（最高+10分）：

| 场景标签 | 匹配逻辑                                   |
|------|----------------------------------------|
| 大菜   | <code>food.level.includes('大菜')</code> |
| 面食   | <code>food.type === '面食系'</code>       |
| 经典   | <code>food.type === '经典系'</code>       |
| 火锅   | <code>food.name.includes('火锅')</code>  |
| 精致   | <code>food.type === '精致系'</code>       |

知识点: `Array.some()` + `String.includes()` 组合判断 — 遍历场景标签数组, 至少一个标签匹配即得分

④ 心情匹配 (最高+10分):

| 心情标签 | 匹配逻辑                                      |
|------|-------------------------------------------|
| 治愈   | <code>food.type === '鲜香系'</code>          |
| 重口   | <code>food.type === '重口系'    '麻辣系'</code> |
| 温暖   | <code>food.name.includes('汤')</code>      |
| 经典   | <code>food.type === '经典系'</code>          |

⑤ 天气匹配 (最高+5分):

| 天气标签 | 匹配逻辑                                      |
|------|-------------------------------------------|
| 汤类   | <code>food.name.includes('汤')</code>      |
| 火锅   | <code>food.name.includes('火锅')</code>     |
| 暖心   | <code>food.type === '鲜香系'    '经典系'</code> |
| 任意   | 直接匹配                                      |

⑥ 安徽美食加分 (+5分):

```
1 if (food.province === '安徽省') {  
2   score += 5;  
3   reasons.push('地道徽菜推荐');  
4 }
```

⑦ 分数上限:

```
1 score = Math.min(score, 100);
```

知识点: `Math.min()` — 确保分数不超过100

### 4.2.3 推荐结果排序

```
1 function recommendFoods(conditions) {
2   const results = foodBosses.map(food => {
3     const match = calculateMatch(food, conditions);
4     return { food, score: match.score, reasons: match.reasons };
5   });
6   results.sort((a, b) => b.score - a.score); // 降序排列
7   return results;
8 }
```

知识点: `Array.sort()` 比较函数 — `b - a` 降序（高分在前），`a - b` 升序

### 4.2.4 推荐结果展示

| 展示元素 | 生成函数                              | 逻辑                                                                              |
|------|-----------------------------------|---------------------------------------------------------------------------------|
| 匹配分数 | —                                 | <code>score</code> 字段, 3rem大字                                                   |
| 星级   | <code>getStarRating(score)</code> | $\geq 90=5$ 星, $\geq 80=4$ 星, $\geq 70=3$ 星, $\geq 60=2$ 星, $<60=1$ 星           |
| 等级文字 | <code>getMatchLevel(score)</code> | $\geq 90=$ 完美匹配, $\geq 80=$ 高度推荐, $\geq 70=$ 值得尝试, $\geq 60=$ 可以考虑, $<60=$ 不太推荐 |
| 推荐理由 | <code>match.reasons</code>        | 列表展示, 蓝色背景卡片, ✓ 前缀                                                              |

## 4.3 LocalStorage 持久化与成就系统

### 4.3.1 数据存储设计

核心知识点: `localStorage` API + `JSON` 序列化 + `Date` 时间戳 + `Set` 去重

```
1 const STORAGE_KEY = 'tongueOfBoss_challenges';
```

记录结构:

```
1 {
2   id: Date.now(),           // 唯一ID（毫秒时间戳）
3   foodId: 1,                // 美食ID（关联 data.js）
}
```

```

4   foodName: "徽州臭鳊鱼",      // 冗余存储（方便查询）
5   date: "2026-06-14",          // ISO日期（YYYY-MM-DD）
6   timestamp: 1718345600000,     // 精确时间戳
7   conditions: {                 // 挑战时的条件
8       mood: 'happy',
9       weather: 'sunny',
10      scene: 'family',
11      taste: 'any'
12  },
13  matchScore: 85,               // 匹配度分数
14  status: 'completed'           // 状态
15  }

```

### CRUD 操作：

| 操作         | 函数                                 | 知识点                                                                                                                                  |
|------------|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| 读取         | <code>getChallengeRecords()</code> | <code>localStorage.getItem()</code> → <code>JSON.parse()</code><br>→ 空数组兜底 <code>\ \  []</code>                                      |
| 新增         | <code>saveChallengeRecord()</code> | <code>getChallengeRecords()</code> → <code>push()</code> →<br><code>JSON.stringify()</code> →<br><code>localStorage.setItem()</code> |
| （无删除/修改操作） | —                                  | 项目不提供删除挑战记录的功能                                                                                                                       |

### 知识点：

- `localStorage.getItem/setItem` — 浏览器本地存储，数据持久化（关闭浏览器不丢失）
- `JSON.parse/stringify` — JS对象与JSON字符串互转
- 短路求值 `data ? JSON.parse(data) : []` — 首次访问（无数据）返回空数组

### 4.3.2 统计数据计算

```

1  function getChallengeStats() {
2      const records = getChallengeRecords();
3      const uniqueFoods = new Set(records.map(r => r.foodId)); // Set去重
4      const dates = [...new Set(records.map(r => r.date))].sort(); // 日期去重+排序
5  }

```

```

6 // 连续天数计算
7 let consecutiveDays = 0;
8 const today = new Date().toISOString().split('T')[0];
9 if (dates.includes(today) || dates[dates.length-1] ===
getYesterday()) {
10     consecutiveDays = 1;
11     for (let i = dates.length - 2; i >= 0; i--) {
12         if (daysDiff(dates[i], dates[i+1]) === 1)
consecutiveDays++;
13         else break;
14     }
15 }
16
17 return {
18     total: foodBosses.length,
19     challenged: uniqueFoods.size,
20     percentage: Math.round((uniqueFoods.size / foodBosses.length)
* 100),
21     consecutiveDays,
22     totalRecords: records.length
23 };
24 }

```

知识点:

- `new Set(array)` — ES6 Set去重（美食ID、挑战日期）
- `[...set]` — 扩展运算符将Set转回数组
- `Array.sort()` — 字符串日期天然可排序（YYYY-MM-DD格式）
- `new Date().toISOString().split('T')[0]` — 获取今日日期字符串
- `Math.round()` — 百分比四舍五入

连续天数算法:

```

1 dates = ['2026-06-10', '2026-06-11', '2026-06-12', '2026-06-14']
2                                     ↑
3                                     从最新日期开始
4 → day14与day12差2天 ≠ 1 → 连续天数=1（只有今天或昨天），断开循环

```



### 4.3.3 成就系统

知识点： 条件判断 + 布尔逻辑 + 动态CSS类切换

7个成就判断逻辑：

| 成就      | 解锁条件     | 判断逻辑                                                                       | 知识点                              |
|---------|----------|----------------------------------------------------------------------------|----------------------------------|
| 美食新手 🏆3 | 首次挑战     | <code>stats.challenged &gt;= 1</code>                                      | 简单比较                             |
| 美食达人 🏆2 | 挑战5道     | <code>stats.challenged &gt;= 5</code>                                      | 简单比较                             |
| 美食专家 🏆1 | 挑战10道    | <code>stats.challenged &gt;= 10</code>                                     | 简单比较                             |
| 舌尖王者 🏆  | 挑战全部15道  | <code>stats.challenged &gt;= 15</code>                                     | 简单比较                             |
| 安徽美食通 🏆 | 完成所有安徽美食 | <code>records.filter(→ province==='安徽省').length &gt;= 10</code>            | <code>Array.filter()</code> 二次过滤 |
| 每日打卡王 🌟 | 连续7天     | <code>stats.consecutiveDays &gt;= 7</code>                                 | 依赖统计计算                           |
| 完美匹配 🎯  | 10次 ≥90分 | <code>records.filter(r =&gt; r.matchScore &gt;= 90).length &gt;= 10</code> | <code>Array.filter()</code>      |

成就渲染：

```
1 const achievementsHtml = achievements.map(a => `  
2   <div class="achievement-card ${a.unlocked ? 'unlocked' :  
   'locked'}">  
3     <div class="achievement-icon">${a.icon}</div>  
4     <div class="achievement-name">${a.name}</div>  
5     <div class="achievement-desc">${a.desc}</div>  
6   </div>  
7 `).join('');
```

CSS状态样式：

```

1  .achievement-card.unlocked {
2      border-color: #FFA500; /* 金色边框 */
3      background: linear-gradient(135deg, #FFF9E6, #FFE5B4); /* 渐变背景 */
4  }
5  .achievement-card.locked {
6      opacity: 0.4; /* 半透明 */
7      filter: grayscale(1); /* 灰度滤镜 */
8  }

```

知识点：CSS `filter: grayscale(1)` — 将元素完全灰度化，配合 `opacity: 0.4` 营造“未解锁”视觉

## 4.4 今日BOSS确定性随机算法

核心知识点： 伪随机种子 + 位运算 + 日期作为种子

```

1  function getTodayBoss() {
2      const today = new Date();
3      const dateString =
4      `${today.getFullYear()}-${today.getMonth()+1}-${today.getDate()}`;
5      // 例如: "2026-6-14"
6
7      // 基于日期字符串生成确定性种子
8      let seed = 0;
9      for (let i = 0; i < dateString.length; i++) {
10         seed = ((seed << 5) - seed) + dateString.charCodeAt(i);
11         seed = seed & seed; // 转换为32位整数
12     }
13
14     const index = Math.abs(seed) % foodBosses.length;
15     return foodBosses[index];
16 }

```

知识点详解：

- `new Date()` — 获取当前日期时间
- `getFullYear()` / `getMonth()` / `getDate()` — 提取年月日
- `dateString.charCodeAt(i)` — 获取字符ASCII码

- `(seed << 5) - seed` — 左移5位等价于 `seed * 31`，经典哈希乘法（如Java的 `hashCode`）
- `seed & seed` — 位与运算，将数字转换为32位有符号整数（防止溢出）
- `Math.abs(seed) % length` — 取绝对值后取模，映射到数组索引范围

算法特性：

- 确定性：同一天 → 相同种子 → 相同索引 → 相同BOSS
- 均匀性：日期字符串微小变化 → 种子大幅变化 → 均匀分布
- 无需存储：纯计算，不需要后端、不需要LocalStorage

为什么叫“BOSS”？ 将美食比作游戏中的BOSS，每天需要挑战一个不同的美食，增加趣味性和游戏化体验。

## 5. 样式系统与设计语言

### 5.1 CSS变量设计系统

```

1  :root {
2      /* 背景色体系 */
3      --bg-primary: #FFFFFF;      /* 白色卡片背景 */
4      --bg-secondary: #F8F9FA;    /* 浅灰页面背景 */
5      --bg-tertiary: #F3F4F6;    /* 更浅灰标签/按钮背景 */
6
7      /* 文字色体系 */
8      --text-primary: #1F2937;    /* 主文字 */
9      --text-secondary: #6B7280;  /* 次级文字 */
10     --text-muted: #9CA3AF;      /* 弱化文字 */
11
12     /* 品牌色体系 */
13     --accent-primary: #E63946;   /* 红色-主品牌色 */
14     --accent-secondary: #F77F00; /* 橙色-挑战系统 */
15     --accent-tertiary: #06AED5;  /* 蓝色-备选 */
16
17     /* 边框体系 */
18     --border-light: #E5E7EB;
19     --border-medium: #D1D5DB;

```

```

20
21  /* 圆角体系 */
22  --radius-sm: 6px;
23  --radius-md: 12px;
24  --radius-lg: 16px;
25
26  /* 阴影体系 */
27  --shadow-sm: 0 1px 3px rgba(0,0,0,0.1); /* 卡片默认 */
28  --shadow-md: 0 4px 6px rgba(0,0,0,0.1); /* 悬浮状态 */
29  --shadow-lg: 0 10px 15px rgba(0,0,0,0.1); /* 强调元素 */
30
31  /* 动效 */
32  --transition: 0.2s ease;
33
34  /* 字体栈 */
35  --font-cn: "PingFang SC", "Microsoft YaHei", "微软雅黑", sans-
    serif;
36  }

```

知识点：CSS 自定义属性 (`--*`)

- 通过 `var(--name)` 引用
- 支持继承和级联
- 可在 `:root` 定义全局变量，也可在局部覆盖
- 配合 `!important` 可实现强制覆盖

## 5.2 通用组件

| 组件    | CSS类                            | 核心样式               |
|-------|---------------------------------|--------------------|
| 按钮-主要 | <code>.btn.btn-primary</code>   | 红色填充，悬浮加深+上移+阴影    |
| 按钮-轮廓 | <code>.btn.btn-outline</code>   | 透明+红边框，悬浮填充红色      |
| 按钮-次要 | <code>.btn.btn-secondary</code> | 灰色填充               |
| 卡片    | <code>.card</code>              | 白底+圆角+细边框，悬浮上移+阴影  |
| 标签    | <code>.tag</code>               | 内联块+圆角胶囊           |
| 标签-主色 | <code>.tag-primary</code>       | 浅红底+红字             |
| 标签-橙色 | <code>.tag-secondary</code>     | 浅橙底+橙字             |
| 导航栏   | <code>.navbar</code>            | sticky定位+白底+底边框+阴影 |
| 页脚    | <code>.footer</code>            | 白底+顶边框+居中+三级文字     |

### 5.3 响应式断点策略

```
1  /* 平板 (≤1024px) – 卡片3列→2列 */
2  @media (max-width: 1024px) { .food-grid { grid-template-columns:
   repeat(2, 1fr); } }
3
4  /* 移动端 (≤768px) – 导航折叠、统计2列 */
5  @media (max-width: 768px) {
6      .nav-links { display: none; }          /* 隐藏导航链接 */
7      .mobile-menu-btn { display: block; }   /* 显示汉堡菜单 */
8      .stats-grid { grid-template-columns: repeat(2, 1fr); }
9      .challenge-cards { grid-template-columns: 1fr; }
10 }
11
12 /* 小屏 (≤640px) – 卡片1列 */
13 @media (max-width: 640px) { .food-grid { grid-template-columns: 1fr; }
   }
```

## 附录A：知识要点索引

### HTML 知识点清单

| 序号 | 知识点                               | 所在文件        | 具体位置/功能                              |
|----|-----------------------------------|-------------|--------------------------------------|
| 1  | 语义化标签                             | 所有页面        | <header>, <nav>, <section>, <footer> |
| 2  | <a> 超链接                           | 所有页面        | 页面导航、卡片跳转                            |
| 3  | <img> 图片 + onerror                | 所有页面        | 美食图片 + 降级处理                          |
| 4  | <input> 表单                        | 所有页面        | 搜索框                                  |
| 5  | <button> 按钮                       | 所有页面        | 搜索、筛选、挑战操作                           |
| 6  | <meta viewport>                   | 所有页面        | 响应式视口                                |
| 7  | <link> 外部CSS                      | 所有页面        | 引入 common.css                        |
| 8  | <script> 外部JS                     | 所有页面        | 引入 data.js / challenge-core.js 等     |
| 9  | URL查询参数 ?id=                      | detail.html | 传递美食ID                               |
| 10 | <svg> / <defs> / <linearGradient> | map.html    | SVG地图渲染                              |

## CSS 知识点清单

| 序号 | 知识点                                            | 所在文件                                   | 具体位置/功能                                         |
|----|------------------------------------------------|----------------------------------------|-------------------------------------------------|
| 1  | CSS变量 (:root / var())                          | common.css                             | 全局设计系统                                          |
| 2  | box-sizing: border-box                         | common.css                             | 盒模型重置                                           |
| 3  | Flexbox (display: flex)                        | common.css + 所有页面                      | 导航栏、按钮组、统计卡片                                    |
| 4  | Grid (display: grid)                           | 所有页面                                   | 美食网格、统计区、成就区                                    |
| 5  | position: sticky                               | common.css + list.html                 | 吸顶导航 + 粘性控制面板                                   |
| 6  | position: absolute/relative                    | index.html + map.html                  | 搜索按钮定位 + Tooltip                                |
| 7  | linear-gradient()                              | 所有页面                                   | Hero、卡片头部、按钮渐变                                  |
| 8  | box-shadow                                     | common.css                             | 三层阴影体系                                          |
| 9  | transition                                     | common.css + 所有页面                      | 悬浮/点击平滑过渡                                       |
| 10 | @keyframes                                     | common.css + map.html + list.html      | searchSlideIn / bounce / anhuiPulse / slideDown |
| 11 | backdrop-filter: blur()                        | list.html + map.html                   | 毛玻璃效果                                           |
| 12 | filter: grayscale()/brightness()/drop-shadow() | common.css + challenge.html + map.html | 成就锁定 / 地图高亮                                     |
| 13 | @media 响应式                                     | 所有页面                                   | 1024px / 768px / 640px 断点                       |
| 14 | ::before / ::after 伪元素                         | map.html                               | Tooltip三角箭头                                     |
| 15 | scroll-behavior: smooth                        | common.css                             | 平滑滚动                                            |
| 16 | :hover / :focus 伪类                             | common.css + 所有页面                      | 交互反馈                                            |
| 17 | display: none/block/flex                       | 所有页面                                   | 元素显隐                                            |
| 18 | paint-order                                    | map.html                               | SVG文字描边顺序                                       |
| 19 | object-fit: cover                              | 所有页面                                   | 图片裁剪适配                                          |
| 20 | overflow: hidden                               | 所有页面                                   | 防止内容溢出                                          |

# JavaScript 知识点清单

| 序号 | 知识点                                                  | 所在文件                                                    | 具体位置/功能         |
|----|------------------------------------------------------|---------------------------------------------------------|-----------------|
| 1  | <code>var / let / const</code>                       | 所有JS文件                                                  | 变量声明            |
| 2  | 箭头函数 <code>() =&gt; {}</code>                        | 所有页面                                                    | 事件回调            |
| 3  | 模板字符串 <code>`...\${}...`</code>                      | 所有页面                                                    | 动态HTML生成        |
| 4  | <code>Array.filter()</code>                          | <code>index.html + list.html + challenge-core.js</code> | 搜索筛选 + 美食过滤     |
| 5  | <code>Array.map()</code>                             | <code>challenge.js + list.html</code>                   | 动态渲染选项/卡片       |
| 6  | <code>Array.find()</code>                            | <code>detail.html</code>                                | 根据ID查找美食        |
| 7  | <code>Array.sort()</code>                            | <code>challenge-core.js</code>                          | 推荐结果排序          |
| 8  | <code>Array.some()</code>                            | <code>index.html + challenge-core.js</code>             | 口味标签匹配 / 条件匹配   |
| 9  | <code>Array.forEach()</code>                         | <code>challenge.js</code>                               | 批量事件绑定          |
| 10 | Set 数据结构                                             | <code>challenge-core.js</code>                          | 去重统计            |
| 11 | 扩展运算符 <code>...</code>                               | <code>challenge-core.js</code>                          | Set转数组          |
| 12 | <code>JSON.parse/stringify</code>                    | <code>challenge-core.js</code>                          | LocalStorage序列化 |
| 13 | <code>localStorage.getItem/setItem</code>            | <code>challenge-core.js</code>                          | 持久化存储           |
| 14 | Date 对象                                              | <code>challenge-core.js</code>                          | 日期计算            |
| 15 | <code>Math.abs/min/round</code>                      | <code>challenge-core.js</code>                          | 数学运算            |
| 16 | <code>String.includes/toLowerCase</code>             | 所有页面                                                    | 字符串模糊匹配         |
| 17 | <code>document.querySelector/querySelectorAll</code> | 所有页面                                                    | DOM查找           |
| 18 | <code>document.getElementById</code>                 | 所有页面                                                    | DOM查找           |
| 19 | <code>element.innerHTML</code>                       | 所有页面                                                    | 动态渲染            |
| 20 | <code>element.classList.add/remove/toggle</code>     | 所有页面                                                    | CSS类控制          |
| 21 | <code>element.getAttribute/setAttribute</code>       | <code>challenge.js + list.html</code>                   | data属性读写        |
| 22 | <code>element.addEventListener</code>                | 所有页面                                                    | 事件绑定            |
| 23 | <code>element.scrollIntoView</code>                  | <code>challenge.js</code>                               | 平滑滚动            |
| 24 | <code>element.contains</code>                        | 所有页面                                                    | 点击外部判断          |
| 25 | <code>element.getBoundingClientRect</code>           | <code>map.html</code>                                   | Tooltip定位       |
| 26 | <code>window.location</code>                         | 所有页面                                                    | 页面跳转            |
| 27 | <code>URLSearchParams</code>                         | <code>detail.html</code>                                | URL参数解析         |
| 28 | <code>window.open</code>                             | 所有页面                                                    | 新标签页搜索          |
| 29 | <code>history.back()</code>                          | <code>detail.html</code>                                | 返回上一页           |
| 30 | <code>encodeURIComponent</code>                      | 所有页面                                                    | URL编码           |
| 31 | DOMContentLoaded 事件                                  | 所有页面                                                    | 初始化入口           |
| 32 | click 事件                                             | 所有页面                                                    | 点击交互            |
| 33 | input 事件                                             | <code>index.html + list.html</code>                     | 实时搜索            |
| 34 | keypress 事件                                          | 所有页面                                                    | 回车搜索            |
| 35 | mouseenter/mousemove/mouseleave                      | <code>map.html</code>                                   | 地图悬浮交互          |
| 36 | <code>event.stopPropagation()</code>                 | <code>index.html</code>                                 | 阻止事件冒泡          |
| 37 | <code>parseInt</code>                                | <code>detail.html</code>                                | 字符串转整数          |
| 38 | 正则表达式 <code>replace()</code>                         | <code>map.html</code>                                   | 省份简称            |
| 39 | 位运算 <code>&lt;&lt; / &amp;</code>                    | <code>challenge-core.js</code>                          | 确定性随机种子         |

| 序号 | 知识点          | 所在文件              | 具体位置/功能    |
|----|--------------|-------------------|------------|
| 40 | charCodeAt() | challenge-core.js | 字符串转ASCII码 |

---

文档完。 本技术文档覆盖了“舌尖BOSS”项目的全部5个页面、所有按钮交互、3个核心系统的算法逻辑，以及所用到的 HTML/CSS/JavaScript 共约70个知识点的详细索引。